

Lab #5

Design and implement hardware and software (PIC18F assembly-program) for inputting a 4-bit number entered via DIP Switches and display the result (0-9) on a Seven-Segment Display.

ECE 3301L Introduction to Microcontrollers Laboratory

Sarah Boyd

Dr. Tamer Omar



**Cal Poly
Pomona**

College of Engineering

Post lab:

Explanation of code:

<https://youtu.be/e-QgOON7Z7g?si=mnebqS0YKtN5oEM4>

Working circuit:

<https://youtu.be/afrs5WzzbQM?si=aYd4lw9jFXoJI9Ay>

Prelab:

Explanation of code:

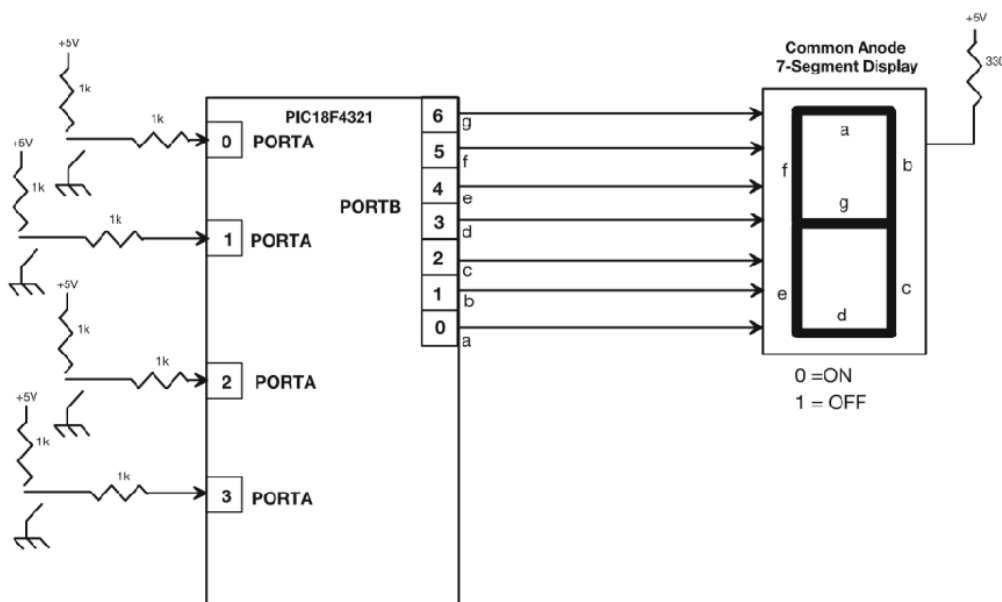
<https://youtu.be/qwmvJ2s0P84?si=4mLS2Hs3aVYjUXSe>

Working circuit: <https://youtu.be/TQTJ5vkMPnc?si=FJiGiWtN6Xj53wte>

Objective: The purpose of this lab is to build hardware and write a PIC18F assembly language program for displaying aBCD number (0-9) on a common cathode seven-segment display based on four DIP switch inputs.

PRELAB:

prelab i) The PIC18F4321 microcontroller is required to input a BCD number (0-9) via four switches connected to bits 0 through 3 of PORTA, and then output the BCD digit to a common-anode seven segment display connected to bits 0 through 6 of PORTB. Write a PIC18F at address 0x70 to accomplish this by storing seven-segment codes in data memory starting at address 0x100. The figure for the lab is provided below:



IMG 1.

4. Equipment, Software, and Components required:

- Microchip's MPLAB C18 Compiler /Debugger
- Parts List
- PICKit3 and PIC18F4321 chip from Microchip
- DIP Switches, Seven-segment Display, and a push button
- Breadboard
- Resistors

- Power Supply

5. Description (corresponding topics covered in the textbook):

Section 8.2, Problem 8.12, Appendix F, Appendix H

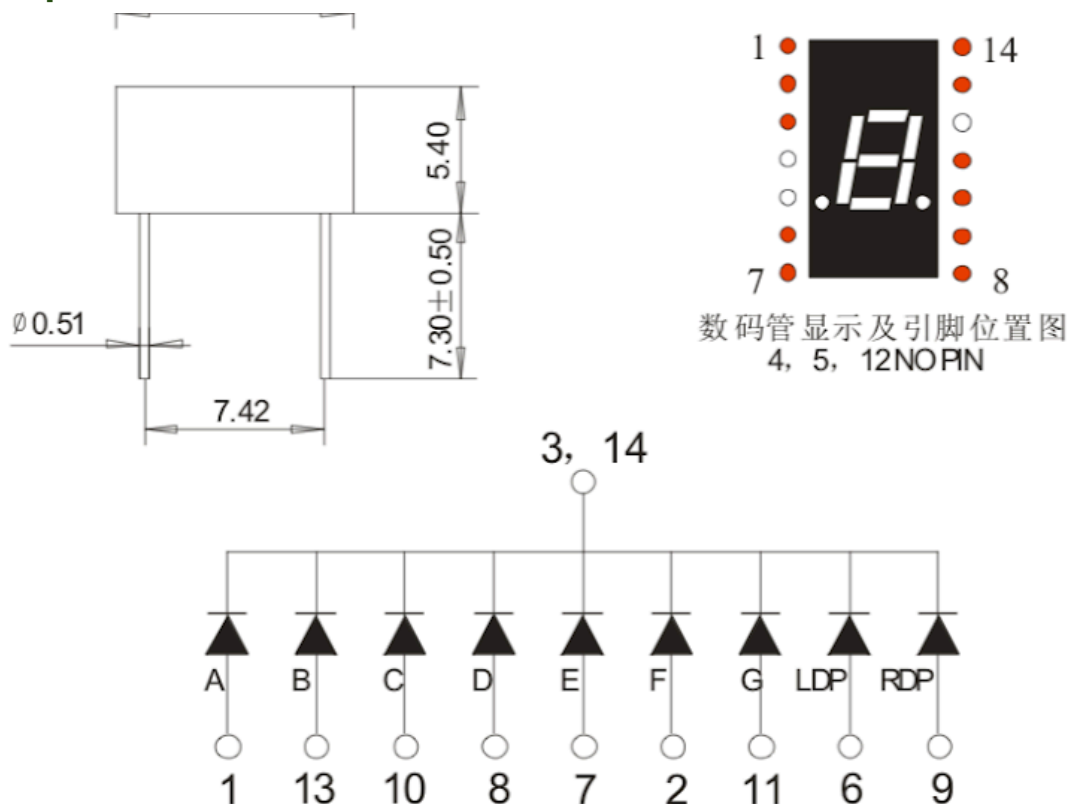
6. Prerequisites:

Example 8.3

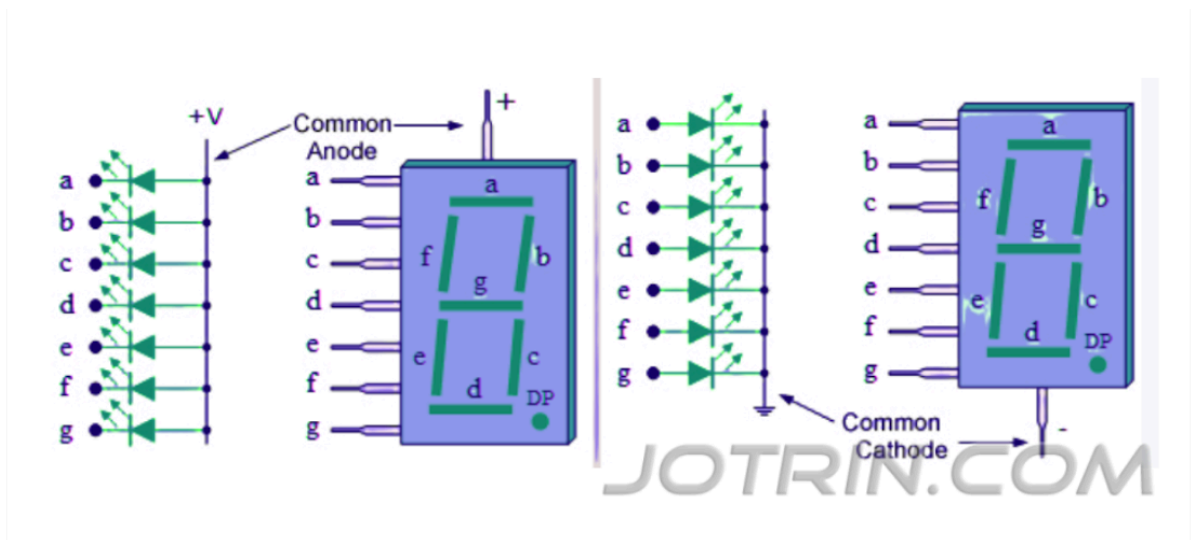
7. Procedure:

- Assemble the PIC18F assembly language program using the MPLAB.
- Download the assembled program into the PIC18F4321 on the breadboard from your Personal Computer or Laptop using the PICKit3 and MPLAB following the steps provided in Appendix H of the book.
- Use the default clock of the PIC18F4321 and connect the appropriate RESET circuit to the PIC18F4321 MCLR pin.
- Connect the DIP switches to port A and the seven-segment display to port B, and demonstrate the lab as a PIC18F4321-based stand-alone system

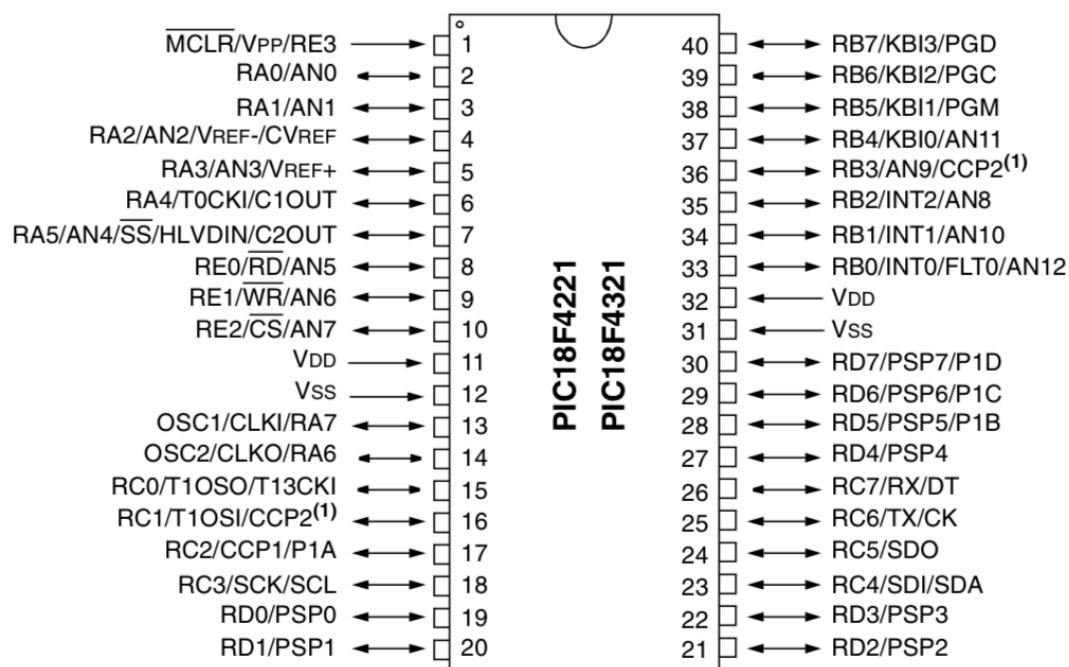
8. Explanation of Hardware



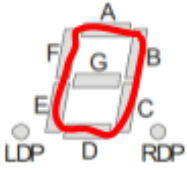
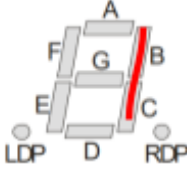
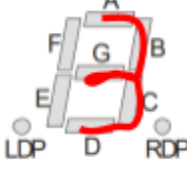
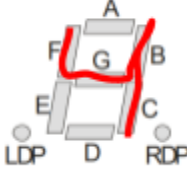
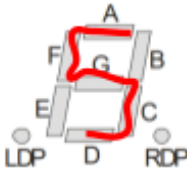
IMG 2. Jameco Electronics 7 segment display common anode, found in the data sheet located here: <https://www.jameco.com/Jameco/Products/ProdDS/24740.pdf>

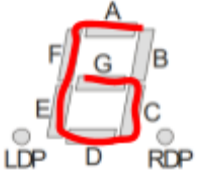
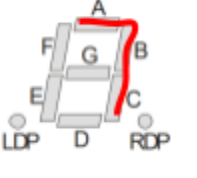
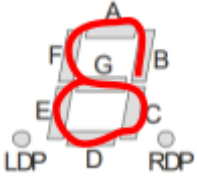
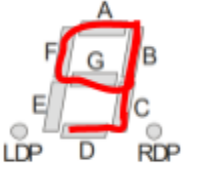


IMG 3. The difference between common anode and cathode, found at <https://www.jotrin.com/technology/details/7-segment-display-comprehensive-guide?srltid=AfmBOoooLUyoBrl-Qj43W4GxEwavxiJgdbri1iNKCf0TdrXQlqfXXcSoy>



IMG 4. Layout of the PIC18F4321, from microchip.com. More information is at the end of this report

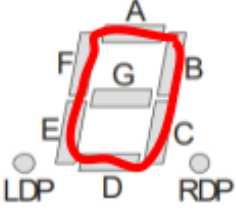
			Port B + corresponding segment								Switch (index value)				Hex value located at index	
			DP	g	f	e		d	c	b						a
Digit	Display	Segments to be connected to ground	7	6	5	4		3	2	1	0	1	2	3	4	
0		a, b, c, d, e, f	1	1	0	0		0	0	0	0	0	0	0	0	0xC0
1		b, c	1	1	1	1		1	0	0	1	0	0	0	1	0xF9
2		a, b, d, e, g	1	0	1	0		0	1	0	0	0	0	1	0	0xA4
3		a, b, c, d, g	1	0	1	1		0	0	0	0	1	1	1	0	0xB0
4		b, g, f, c	1	0	0	1		1	0	0	1	0	1	0	0	0x99
5		a, f, g, c, d	1	0	0	1		0	0	1	0	0	1	0	1	0x92

6		a, f, e, d, c, g	1	0	0	0		0	0	1	0	0	1	1	0	0x82
7		a, b, c	1	1	1	1		1	0	0	0	0	1	1	1	0xF8
8		a, b, c, d, e, f, g	1	0	0	0		0	0	0	0	1	0	0	0	0x80
9		a, b, c, d, f, g	1	0	0	1		0	0	0	0	1	0	0	1	0x90

Here is an example about how to get a number on the seven segment display from four inputs from four switches.

Example 1- zero

Note that the binary values needed to turn on the LEDS were pre-determined and stored in an array. For example, the binary value needed to display zero was found by determining which segments need to be turned on.

Digit	Display	Segments to be connected to ground
0		a, b, c, d, e, f

	Decimal point	g	f	e		d	c	b	a
Bit # on port B	7	6	5	4		3	2	1	0
Value	1	1	0	0		0	0	0	0

Again, note that connecting the segment to ground and giving it the value 0 turns on the LED. The decimal point is automatically a 1 because we don't want it displayed, and g is excluded from the segments we want on. Therefore, the binary value to turn on 0 would be 1100 0000.

Bit	1	1	0	0	0	0	0	0
Place	$2^7 = 128$	$2^6 = 64$	$2^5 = 32$	$2^4 = 16$	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$

$128 + 64 = 192$ (decimal value)

To find our expected hex result, we divide by 16 repetitively until we get a remainder of 0, where the first quotient is the smallest digit.

$$192 / 16 = 12$$

12 in hex is C, or 0xC0.

This process was repeated for values 0-15, and stored in an array.

Step 1: Get the switch inputs

When the switch is open, it equals 1 and has an output going to the Pic18F of 5 volts. When the switch is closed, it equals 0 and has an output to the Pic18F of 0 volts.

Switch 1	Switch 2	Switch 3	Switch 4
Closed	Closed	Closed	Closed
0	0	0	0

The input goes into port A, giving the port A input bits a value.

Bit	0	1	2	3
input	0	0	0	0

Port A, in total, equals

7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0

Step 2: get the 4 lower bits from port A

The switch values can give us the index, the location of the value we want to display. Since the switch values are stored in the lower 4 bits of port A

“Port A & 0x0f”

This section of code masks the higher bits, and we just keep lower 4 bits (0000). This is the decimal value 0

Step 3: Use the value from the lower 4 bits as an index in the table

“LATB = sseg [PORTA & 0x0f]”

“LATB = sseg [0]”

We want to look at the value at place 0 in the array that was previously declared.

“const char sseg [16]= {0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8, 0x80, 0x90, 0x88, 0x83, 0xC6, 0xA1, 0x86, 0x8E};”

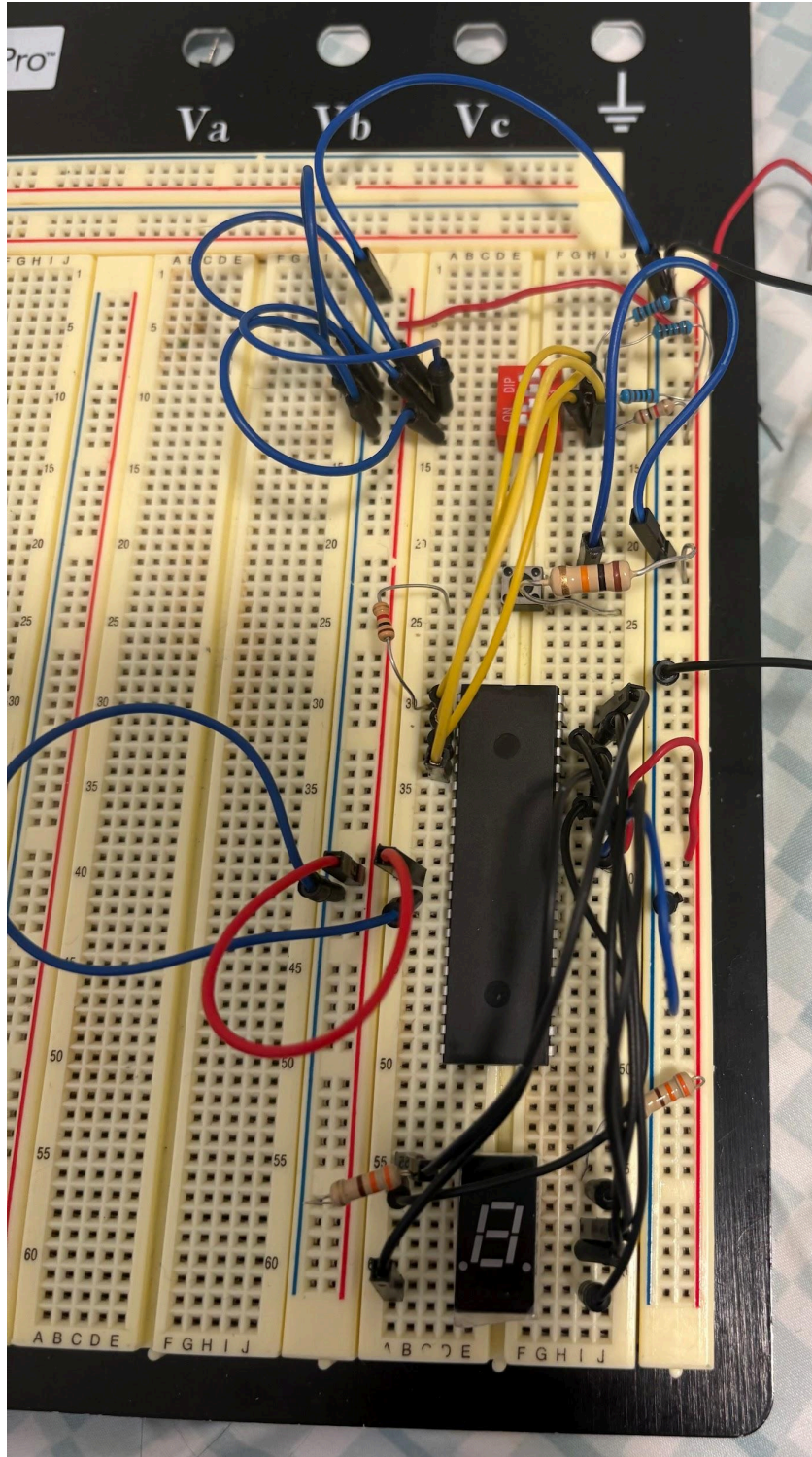
The value at index 0 is 0xC0, so LATB = 0xC0, or LATB = 1100 0000.

Step 4. Display the value at the index

So port B equals

Bit	7	6	5	4	3	2	1	0
Corresponding segment	DP	g	f	e	d	c	b	a
	1	1	0	0	0	0	0	0

Output bits 5, 4, 3, 2, 1, and 0 are set to 0, meaning that segments a, b, c, d, e, f get the input of 0 and are thus connected to ground, and turn on.



IMG 5. *Circuit with common anode 7 segment display, with 330 resistors connected to 5v.*

While actually assembling the circuit, I had to ensure that I used the proper 7 segment display. One mistake I made was to use a common cathode 7 segment display (all the

positive terminals, cathodes, of the 8 leds are interconnected, leaving the negative terminals isolated). This actually would mean that the leds would have an inputs of ground and an output of values, which is wrong because the leds should have inputs of high (5v) depending on the switches, and connecting the segment to ground turns on the segment because all segments are already connected to the volts. To fix this, I made sure to either use a common anode 7 segment display or to adjust the circuit in some way to compensate for this. The circuit can also be adjusted by moving the 330 ohm for the common cathode display to be ground instead of the power line. Another mistake I made was somehow getting negative voltage across my 330 ohm and ground- this indicates that I misconfigured my inputs and outputs, and I adjusted my code accordingly.

As far as selecting a resistor value for the power inputs to the display, 330 ohm resistor limits the current enough to protect the LEDs while still providing enough brightness. Recall that the seven segment display is directly connected to the supply voltage.

Works cited

- *Microchip*, ww1.microchip.com/downloads/en/DeviceDoc/39689b.pdf. Accessed July 2025.
- *Www.jotrin.com*, Jotrin |. "The Ultimate Guide to 7 Segment Display: Pinout, Working, and Datasheet." Jotrin Electronics, www.jotrin.com/technology/details/7-segment-display-comprehensive-guide?srsId=AfmBOoooLUyoBrI-Qj43W4GxEwavxiJgdbr1iNKcf0TdrXQlqfXXcSoy. Accessed 16 July 2025.